



Relatório do Projeto

Parte 2

Nome do Integrante	RA
Luis Felipe Santos do Nascimento	10420572

Descoberta Musical por Grafos: modelagem tripartida de álbuns, artistas e gêneros

Integrante: Luis Felipe Santos do Nascimento – RA 10420572 – 10420572@mackenzista.com.br

Ciência da Computação – Turma 6ºD – Teoria dos Grafos – Prof. Dr. Ivan Carlos Alcântara de Oliveira
Faculdade de Computação e Informática – Universidade Presbiteriana Mackenzie – São Paulo, setembro de 2026

Título da aplicação: Descoberta Musical por Grafos – Álbuns, Artistas e Gêneros. **Repositório:**
<https://github.com/luisfelipesdn12/grafos-musical-tg>

RESUMO

Este relatório apresenta a segunda parte do projeto da disciplina de Teoria dos Grafos: a modelagem, como grafo, de um problema real de descoberta musical. Atendendo ao parecer da Parte 1, o escopo foi reduzido e os usuários foram retirados do modelo; o grafo passou a conter exclusivamente álbuns, artistas e gêneros, obtidos de metadados públicos da base colaborativa MusicBrainz (MUSICBRAINZ, [2026]). A partir de 16 artistas semente – oito do R&B alternativo/hip-hop norte-americano e oito da Música Popular Brasileira (MPB) – foi construído um grafo tripartido, não orientado e ponderado nas arestas (tipo 2), com 109 vértices (16 artistas, 56 álbuns e 37 gêneros) e 349 arestas. O peso de cada aresta é um custo de descoberta, definido como 1 menos a afinidade entre as entidades, em que a afinidade de gênero é derivada dos votos da comunidade do MusicBrainz. O grafo é representado por matriz de adjacência em Python, manipulado por uma aplicação de console com menu de dez opções (leitura e gravação de arquivo, inserção e remoção de vértices e arestas, exibição e conexidade) e validado por 22 testes automatizados. A análise mostra um grafo conexo, de densidade 0,059, em que gêneros como pop, funk, soul funcionam como pontes entre as duas cenas musicais. A próxima etapa aplicará o algoritmo de Dijkstra para encontrar a trilha de descoberta de menor custo entre dois álbuns.

Palavras-chave: teoria dos grafos; grafo tripartido; matriz de adjacência; descoberta musical; MusicBrainz.

1 INTRODUÇÃO

1.1 Contextualização

A forma de consumir música mudou profundamente com as plataformas de streaming. O acesso a catálogos com dezenas de milhões de faixas ampliou a oferta, mas também tornou a descoberta de novos artistas dependente de sistemas de recomendação e de listas automáticas. Datta, Knox e Bronnenberg (2018) mostram que a adoção do streaming altera de forma mensurável o que as pessoas ouvem e a variedade do que descobrem. Nesse cenário, entender como álbuns,



artistas e gêneros se relacionam é um problema computacional concreto: as relações entre essas entidades formam naturalmente uma rede, e a Teoria dos Grafos oferece o vocabulário e os algoritmos para representá-la e percorrê-la (BONDY; MURTY, 2010).

Este projeto é a versão reduzida, para a disciplina, do Trabalho de Conclusão de Curso do autor, que estuda uma plataforma social de álbuns com usuários reais. Aqui fica apenas a camada de conteúdo – álbum, artista e gênero –, que é pública, não envolve dados pessoais e já basta para um problema de grafos bem delimitado.

1.2 Justificativa

Metadados musicais públicos, como os do MusicBrainz, registram não só quem gravou cada álbum, mas também como a comunidade classifica álbuns e artistas em gêneros, por meio de votos. Essa informação permite medir quão fortemente um álbum pertence a um gênero e, portanto, quão "perto" dois álbuns estão um do outro passando por gêneros e artistas em comum. Um grafo ponderado com esses dados permite responder a perguntas de descoberta – por exemplo, quais gêneros ligam o R&B contemporâneo à MPB dos anos 1970 – sem depender de histórico de usuários, o que evita o problema de partida a frio (cold start) dos sistemas de recomendação baseados em interação (LIKA; KOLOMVATSOS; HADJIEFTHYMIADES, 2014) e dispensa o tratamento de dados pessoais regulado pela Lei Geral de Proteção de Dados (BRASIL, 2018).

1.3 Objetivo

Objetivo geral: modelar, como grafo tripartido ponderado, as relações entre álbuns, artistas e gêneros de duas cenas musicais reais e desenvolver um protótipo que carregue, edite, exiba e analise esse grafo, preparando a função central do próximo bimestre: a "trilha de descoberta" de menor custo entre dois álbuns.

Objetivos específicos:

- coletar, da API pública do MusicBrainz, os álbuns de estúdio e os votos de gênero de 16 artistas semente, sem nenhum dado de usuário;
- definir com precisão os tipos de vértice, os tipos de aresta e o significado do peso;
- gerar o arquivo grafo.txt (tipo 2) com pelo menos 80 vértices e 200 arestas;
- implementar a classe do grafo por matriz de adjacência e a aplicação com menu a–j;
- testar a aplicação e analisar a estrutura do grafo (graus, densidade, conexidade e gêneros que ligam as cenas).

1.4 Atendimento ao parecer da Parte 1

A proposta da Parte 1 recebeu nota 8,0, com a recomendação de reduzir o escopo, retirar os usuários e trabalhar "exclusivamente com álbuns, artistas e gêneros". A Tabela 1 relaciona cada ponto do parecer à resposta dada nesta etapa.



Tabela 1 – Pontos do parecer da Parte 1 e como foram atendidos

Ponto do parecer	Resposta na Parte 2	Onde
Escopo amplo demais (plataforma social, recomendação, comunidades, clusterização)	Escopo reduzido a um problema central: modelar a rede álbum–artista–gênero e, na próxima etapa, encontrar a trilha de descoberta de menor custo (Dijkstra).	Seções 1.3 e 7
Evitar coleta de usuários (procedimentos éticos)	Usuários retirados do modelo; apenas metadados públicos de álbuns, artistas e gêneros do MusicBrainz.	Seções 4 e 5.1
Definir precisamente os tipos de vértice e de aresta	Três tipos de vértice com prefixo no rótulo e três tipos de aresta com semântica própria; arestas entre vértices do mesmo tipo são proibidas.	Seção 5.2, Tabelas 3 e 4
Explicar o significado dos pesos	Peso = custo de descoberta = $1 - \text{afinidade}$; afinidade calculada a partir dos votos de gênero, com exemplo numérico real.	Seção 5.2, Tabela 5
Garantir ≥ 80 vértices e ≥ 200 arestas	Grafo real com 109 vértices e 349 arestas, verificado por teste automático.	Seções 5.5 e 5.6
Contextualização extensa em relação à delimitação computacional	Contextualização reduzida e voltada ao problema computacional.	Seção 1.1

Fonte: autoria própria.

2 FUNDAMENTAÇÃO TEÓRICA

Um **grafo** $G = (V, E)$ é formado por um conjunto de vértices V e um conjunto de arestas E , em que cada aresta liga um par de vértices (BONDY; MURTY, 2010; DIESTEL, 2010). O grafo é **não orientado** quando a aresta $\{v, w\}$ não tem sentido, isto é, liga v a w e w a v ao mesmo tempo; é **ponderado nas arestas** quando cada aresta recebe um valor numérico (peso ou custo). Na classificação usada na disciplina, um grafo não orientado com peso nas arestas é do **tipo 2**, que é o tipo adotado neste projeto.

Um grafo é **k-partido** quando seus vértices podem ser divididos em k conjuntos disjuntos de modo que nenhuma aresta ligue dois vértices do mesmo conjunto; com $k = 3$, diz-se **tripartido**. Redes com vários tipos de vértice e de aresta também são estudadas como redes de informação heterogêneas, em que o tipo de cada elemento carrega significado e orienta a análise (SHI et al., 2017).

O **grau** $d(v)$ de um vértice é o número de arestas incidentes a ele; em um grafo não orientado, a soma dos graus é $2m$, em que m é o número de arestas, de modo que o grau médio é $2m/n$. A **densidade** de um grafo simples é a razão entre o número de arestas existentes e o máximo possível, $2m/(n(n - 1))$. Um **caminho** é uma sequência de vértices em que vértices consecutivos são adjacentes; seu comprimento é o número de arestas e, em grafos ponderados, seu custo é a soma dos pesos. A **distância** entre dois vértices é o comprimento do menor caminho entre eles, e o **diâmetro** é a maior distância entre pares de vértices.

Um grafo não orientado é **conexo** quando existe caminho entre todo par de vértices; caso contrário, divide-se em **componentes conexas**, subgrafos conexos maximais. As componentes podem ser encontradas por **busca em largura** (BFS): a partir de um vértice não visitado, visitam-se todos os alcançáveis, camada a camada, usando uma fila; cada busca iniciada em um vértice ainda não visitado revela uma nova componente (CORMEN et al., 2001; SZWARCFITER, 1988). As categorias de conexidade $C0$ a $C3$, o algoritmo FCONEX e o **grafo reduzido** dizem respeito a grafos orientados, em que a alcançabilidade depende do sentido das arestas; por isso não se aplicam a este grafo, que é não orientado.



Um grafo pode ser armazenado por **matriz de adjacência** – uma matriz $n \times n$ em que a posição (i, j) indica a aresta entre i e j – ou por **lista de adjacência**, que guarda para cada vértice apenas os seus vizinhos. A matriz ocupa $O(n^2)$ de memória, mas responde em $O(1)$ se uma aresta existe e qual o seu peso; a lista ocupa $O(n + m)$ e é mais econômica em grafos esparsos (CORMEN et al., 2001). Em um grafo não orientado a matriz é simétrica. Com n na casa das centenas, a matriz cabe com folga na memória e segue a estrutura trabalhada em aula, razão pela qual foi escolhida.

Por fim, o problema do **caminho mínimo** busca o caminho de menor custo entre dois vértices. O algoritmo de Dijkstra o resolve quando todos os pesos são não negativos (CORMEN et al., 2001). Esse requisito orientou a definição do peso deste projeto como custo em $[0, 1]$, e não como similaridade: quanto menor o peso, mais próximas estão as entidades, e um caminho de custo baixo corresponde a uma sequência de passos de descoberta "naturais".

3 TRABALHOS RELACIONADOS

Grafos bipartidos e multipartidos são a base de vários sistemas de recomendação em larga escala. O **GraphJet**, do Twitter, mantém em memória um grafo bipartido de interações entre usuários e tweets e gera recomendações em tempo real por meio de passeios aleatórios nesse grafo (SHARMA et al., 2016). O **Pixie**, do Pinterest, também usa passeios aleatórios com reinício sobre um grafo bipartido de pins e quadros para recomendar conteúdo a centenas de milhões de usuários (EKSOMBATCHAI et al., 2018). Já o **PinSage** combina essa estrutura com redes neurais convolucionais em grafos, aprendendo representações dos itens a partir da vizinhança amostrada por passeios aleatórios (YING et al., 2018).

No estudo da estrutura de grandes redes, Ugander et al. (2011) analisaram o grafo social do Facebook – distribuição de graus, distâncias e conexidade – e mostraram que quase todos os usuários ativos pertencem a uma única componente gigante. Blondel et al. (2008) propuseram o método de Louvain, que detecta comunidades maximizando a modularidade e é aplicável a redes com milhões de vértices. Shi et al. (2017) revisam a análise de redes de informação heterogêneas, nas quais vértices e arestas de tipos diferentes – como autor, obra e tema – são tratados de forma distinta, exatamente o caso do grafo álbum–artista–gênero deste projeto.

No domínio musical, Van den Oord, Dieleman e Schrauwen (2013) recomendam músicas a partir do conteúdo de áudio para contornar a falta de histórico de uso; o presente projeto persegue o mesmo objetivo – recomendar sem dados de usuário – usando, em vez do áudio, a estrutura de metadados. A fonte de dados é o MusicBrainz, enciclopédia musical aberta e colaborativa mantida pela MetaBrainz Foundation, que expõe artistas, grupos de lançamento (álbuns) e gêneros votados pela comunidade por meio de uma API web pública (MUSICBRAINZ, [2026]). Diferentemente dos sistemas citados, que operam com bilhões de arestas e dados de interação, este trabalho é um protótipo didático, com dados exclusivamente públicos e foco em uma modelagem explícita e interpretável dos pesos.

4 METODOLOGIA

O trabalho foi organizado em oito etapas, todas automatizadas por scripts do repositório, de modo que qualquer resultado deste relatório pode ser reproduzido:

- **Seleção da semente:** 16 artistas escolhidos pelo autor, oito de R&B alternativo/hip-hop e oito de MPB, registrados em `coleta/artistas_semente.txt` com a cena de cada um. Quando a busca



por nome encontra um homônimo (Steve Lacy, o saxofonista de jazz, em vez do músico de R&B), o identificador (MBID) é fixado no arquivo.

- **Coleta:** coleta/coletar_musicbrainz.py consulta a API do MusicBrainz, respeitando o limite de uma requisição por segundo, e grava um JSON por artista em dados/brutos, com os gêneros votados do artista e de cada álbum de estúdio.
- **Filtragem:** apenas grupos de lançamento do tipo primário "Album" sem tipo secundário (excluem-se coletâneas, discos ao vivo e trilhas); por entidade, apenas os cinco gêneros mais votados; por artista, até quatro álbuns com gênero, os mais votados primeiro.
- **Construção:** coleta/montar_grafo.py cria os vértices e as arestas, calcula os pesos, descarta gêneros ligados a menos de duas entidades e grava dados/grrafo.txt, dados/vertices.csv e dados/arestas.csv.
- **Validação:** o script verifica $n \geq 80$ e $m \geq 200$ e os testes conferem o tipo 2 e a tripartição; a leitura com a própria classe da aplicação garante que o arquivo é válido.
- **Aplicação:** src/app.py oferece o menu a–j sobre a classe GrafoMatrizPonderado.
- **Testes:** testes automatizados com unittest e roteiros de uso do menu, registrados em imagens (ao menos dois testes por opção).
- **Visualização e análise:** visualizacao/visualizar.py desenha o grafo (Plotly) e o exporta para o Gephi; ferramentas/gerar_relatorio.py calcula as estatísticas e gera este relatório.

5 RESULTADOS PARCIAIS

5.1 Coleta dos dados

A coleta foi feita em 28/09/2026 pela API web do MusicBrainz, que devolve JSON e não exige autenticação. Para cada artista, o coletor (i) busca o identificador pelo nome, (ii) obtém os gêneros votados do artista e (iii) pagina os grupos de lançamento do tipo álbum, pedindo os gêneros de cada um. O campo de gêneros da API contém apenas as etiquetas (tags) que constam na lista oficial de gêneros do MusicBrainz, cada uma com o número de votos recebidos. Ao todo foram obtidos 287 álbuns de estúdio, dos quais 232 têm ao menos um gênero votado; após a filtragem, 56 álbuns entraram no grafo. Os artistas Frank Ocean, Omar Apollo, SZA, Steve Lacy têm menos de quatro álbuns com gênero votado e, por isso, contribuem com menos álbuns. A Tabela 2 resume a coleta por artista.

Tabela 2 – Resumo da coleta por artista semente

Artista	Cena	Álbuns de estúdio	Com gênero votado	No grafo	Gêneros do artista
Frank Ocean	R&B/hip-hop	3	2	2	4
Tyler, The Creator	R&B/hip-hop	12	9	4	12
Daniel Caesar	R&B/hip-hop	5	4	4	3
Omar Apollo	R&B/hip-hop	2	2	2	2
SZA	R&B/hip-hop	2	2	2	8
Steve Lacy	R&B/hip-hop	3	2	2	18
Kendrick Lamar	R&B/hip-hop	9	6	4	16
Anderson .Paak	R&B/hip-hop	7	6	4	8
Caetano Veloso	MPB	33	26	4	5
Gilberto Gil	MPB	44	28	4	6
Chico Buarque	MPB	27	24	4	5
Gal Costa	MPB	31	29	4	5
Milton Nascimento	MPB	35	27	4	7
Jorge Ben Jor	MPB	29	25	4	5



Artista	Cena	Álbuns de estúdio	Com gênero votado	No grafo	Gêneros do artista
Tim Maia	MPB	34	29	4	6
Os Mutantes	MPB	11	11	4	6
Total		287	232	56	

Fonte: autoria própria, dados do MusicBrainz.

5.2 Modelagem do problema como grafo

O problema é modelado como um grafo **tripartido, não orientado e ponderado nas arestas (tipo 2)**. As três partes são os artistas, os álbuns e os gêneros; o tipo de cada vértice fica codificado no próprio rótulo, por um prefixo, e a Tabela 3 apresenta essa definição com as quantidades reais do grafo.

Tabela 3 – Tipos de vértice

Tipo	Prefixo do rótulo	Significado	Exemplo	Quantidade
Artista	[ART]	Artista semente (pessoa ou grupo) de uma das duas cenas	[ART] Gal Costa	16
Álbum	[ALB]	Álbum de estúdio do artista (grupo de lançamento), no formato título — artista	[ALB] Blonde — Frank Ocean	56
Gênero	[GEN]	Gênero musical oficial do MusicBrainz ligado a pelo menos duas entidades	[GEN] tropicália	37
Total				109

Fonte: autoria própria.

Há três tipos de aresta, um para cada par de partes (Tabela 4). O peso de toda aresta é um **custo de descoberta**: $\text{peso} = 1 - a$, arredondado a duas casas, em que $a \in (0, 1]$ é a **afinidade** entre as duas entidades. Para as arestas de gênero, a afinidade é relativa ao gênero mais votado da própria entidade: $a(e, g) = \text{votos}(e, g) / \max \text{votos}(e, g')$, considerando apenas os cinco gêneros mais votados da entidade e desempatando por ordem alfabética.

Tabela 4 – Tipos de aresta, semântica e peso

Relação	Semântica	Afinidade a	Peso (custo)	Quantidade
Álbum – Artista	Autoria: o álbum foi gravado pelo artista	1,0 (sempre)	0,00	56
Álbum – Gênero	A comunidade classifica o álbum no gênero	$\text{votos}(\text{álbum}, g) / \text{máx. de votos do álbum}$	$1 - a \in [0; 1)$	227
Artista – Gênero	O artista transita pelo gênero ao longo da carreira	$\text{votos}(\text{artista}, g) / \text{máx. de votos do artista}$	$1 - a \in [0; 1)$	66
Total				349

Fonte: autoria própria.

Exemplo numérico com dados reais. A Tabela 5 mostra o cálculo para o álbum Blonde (Frank Ocean, 2016). O gênero mais votado dá o denominador; cada gênero recebe afinidade proporcional aos seus votos e o peso da aresta é o complemento dessa afinidade.

Tabela 5 – Cálculo dos pesos das arestas de gênero do álbum Blonde

Gênero	Votos	Afinidade	Peso	Situação
neo soul	3	$3/3 = 1,00$	$1 - 1,00 = 0,00$	aresta no grafo
alternative r&b	2	$2/3 = 0,67$	$1 - 0,67 = 0,33$	aresta no grafo
art pop	2	$2/3 = 0,67$	$1 - 0,67 = 0,33$	aresta no grafo
contemporary r&b	1	$1/3 = 0,33$	$1 - 0,33 = 0,67$	aresta no grafo

Fonte: autoria própria, dados do MusicBrainz.

Assim, o caminho que vai de Blonde ao gênero neo soul tem custo 0,00 (é a classificação mais forte do álbum), enquanto a ligação com contemporary r&b custa 0,67. A aresta de autoria



Blonde – Frank Ocean tem custo 0,00, a ligação mais forte possível. O mesmo cálculo vale para artistas: com o artista Tim Maia, todos os gêneros votados têm o mesmo número de votos e, por isso, todas as suas arestas de gênero recebem custo 0,00 – um efeito da escassez de votos discutido na Seção 5.5.

Justificativas de modelagem.

- **Não orientado:** as relações de autoria e de classificação são simétricas – se o álbum pertence ao gênero, o gênero contém o álbum –, e a descoberta pode percorrê-las nos dois sentidos.
- **Peso na aresta (e não no vértice):** o que varia é a força da ligação entre duas entidades (quantos votos a comunidade deu àquele gênero para aquele álbum), e não uma propriedade isolada de um vértice. Por isso o tipo é o 2.
- **Tripartido:** duas entidades do mesmo tipo nunca se ligam diretamente; a proximidade entre dois álbuns surge por meio de um artista ou de um gênero em comum. Isso deixa cada aresta com uma semântica única e verificável, que a classe do grafo impõe ao recusar arestas entre vértices do mesmo tipo.
- **Custo = 1 – afinidade:** o custo é não negativo e menor para ligações mais fortes, requisito do algoritmo de Dijkstra, que será aplicado na próxima etapa para achar a trilha de descoberta de menor custo.
- **"Sem aresta" = None:** como custo 0,00 é um peso válido (autoria, gênero mais votado), o valor 0 não pode significar ausência de aresta na matriz; usa-se o valor None do Python.
- **Filtros:** limitar a cinco gêneros por entidade e exigir que o gênero ligue ao menos duas entidades elimina etiquetas isoladas, que não contribuiriam para nenhum caminho entre entidades diferentes.

5.3 Representação computacional e estruturas de dados

O grafo é implementado pela classe GrafoMatrizPonderado (src/grafos_matriz.py), construída a partir da classe de matriz de adjacência vista em aula. Suas estruturas são: n e m (números de vértices e de arestas); adj , uma lista de listas $n \times n$ de números reais em que $adj[v][w]$ guarda o peso da aresta $\{v, w\}$ ou None quando ela não existe (a matriz é mantida simétrica); e $rotulos$, um vetor de textos em que $rotulos[v]$ é o rótulo do vértice v , com o prefixo do tipo. A Tabela 6 relaciona as operações.

Tabela 6 – Operações da classe GrafoMatrizPonderado

Método	O que faz
<code>insereV(rotulo)</code>	Acrescenta uma linha e uma coluna com None e devolve o novo índice.
<code>removeV(v)</code>	Remove a linha e a coluna de v e todas as arestas incidentes; os vértices seguintes têm o índice decrementado.
<code>insereA(v, w, peso)</code>	Grava o peso em $adj[v][w]$ e $adj[w][v]$; recusa laço, aresta repetida, índice inválido, peso fora de $[0, 1]$ e aresta entre vértices do mesmo tipo.
<code>removeA(v, w)</code>	Volta $adj[v][w]$ e $adj[w][v]$ para None.
<code>vizinhos(v)</code> , <code>grau(v)</code> , <code>tipoVertice(v)</code>	Consultas: vizinhos com peso, grau e tipo (pelo prefixo do rótulo).
<code>componentes()</code> , <code>ehConexo()</code>	Componentes conexas por busca em largura; conexo se houver uma só.
<code>textoLista()</code> , <code>textoMatriz()</code>	Exibição como lista de adjacência com rótulos e pesos, ou como matriz compacta.

Fonte: autoria própria.

O arquivo dados/grafos.txt segue o formato do enunciado: na primeira linha o tipo do grafo (2); na segunda, n ; em seguida, n linhas no formato índice "rótulo"; depois, m ; e, por fim, m linhas "v w



peso", com cada aresta gravada uma única vez. O módulo `src/arquivo_grafo.py` lê o arquivo (recusando outros tipos com mensagem), grava o grafo da memória no mesmo formato e formata o conteúdo em tabelas para a opção g do menu. A aplicação `src/app.py` implementa o menu da Tabela 7; entradas inválidas – índice fora do intervalo, peso não numérico, aresta inexistente, fim da entrada – geram mensagens, nunca exceções.

Tabela 7 – Opções do menu da aplicação

Opção	Função	Implementação
a	Ler dados do arquivo grafo.txt	<code>arquivo_grafo.ler</code>
b	Gravar dados no arquivo grafo.txt	<code>arquivo_grafo.gravar</code>
c	Inserir vértice (tipo e nome)	<code>insereV</code>
d	Inserir aresta (v, w, peso)	<code>insereA</code>
e	Remover vértice	<code>removeV</code>
f	Remover aresta	<code>removeA</code>
g	Mostrar conteúdo do arquivo	<code>arquivo_grafo.formatarConteudo</code>
h	Mostrar grafo (lista ou matriz)	<code>textoLista / textoMatriz</code>
i	Apresentar a conectividade (componentes)	<code>componentes (BFS)</code>
j	Encerrar a aplicação	—

Fonte: autoria própria.

Organização do GitHub. O repositório público (<https://github.com/luisfelipesdn12/grafo-musical-tg>) está organizado em pastas por responsabilidade (Tabela 8). Todo arquivo-fonte começa com um cabeçalho que identifica o projeto, o integrante (nome e RA), a síntese do conteúdo e o histórico de alterações. As bibliotecas externas (NetworkX, Plotly, python-docx) são usadas apenas para visualização e geração do relatório; nenhum algoritmo da aplicação depende delas.

Tabela 8 – Organização do repositório

Caminho	Conteúdo
<code>src/grafo_matriz.py</code>	Classe <code>GrafoMatrizPonderado</code> (matriz de adjacência, tipo 2)
<code>src/arquivo_grafo.py</code>	Leitura, gravação e formatação do <code>grafo.txt</code>
<code>src/app.py</code>	Aplicação de console com o menu a–j
<code>coleta/</code>	Coletor do MusicBrainz, montagem do grafo e lista de artistas semente
<code>dados/</code>	<code>grafo.txt</code> , <code>vertices.csv</code> , <code>arestas.csv</code> e os JSON brutos da coleta
<code>testes/</code>	Testes automatizados (unittest)
<code>visualizacao/</code>	Figura (Plotly), HTML interativo e <code>grafo.gexf</code> para o Gephi
<code>ferramentas/</code>	Geradores dos prints de teste e deste relatório
<code>relatorio/</code>	Relatório (.docx) e figuras

Fonte: autoria própria.

Histórico de evolução. O desenvolvimento foi registrado em commits incrementais, listados na Tabela 9 (o commit deste relatório é o último).



Tabela 9 – Histórico de commits do repositório

Data	Commit	Descrição
28/09/2026	326bd9f	Estrutura inicial do repositório
28/09/2026	0df4c71	Classe GrafoMatrizPonderado (matriz de adjacência, tipo 2)
28/09/2026	0100445	Cabeçalho no arquivo de testes
28/09/2026	849f249	Leitura e gravação do grafo.txt
28/09/2026	e909b03	Coleta MusicBrainz e grafo.txt real
28/09/2026	bd0080a	Aplicação com menu de opções a-j
28/09/2026	3ed33c5	Corrige colisão de rótulo de álbuns com mesmo título
28/09/2026	a84584c	Corrige EOFError no laço principal, captura de erro em op_b e valida opção de op_h
28/09/2026	0f0f06c	Visualização Plotly e export GEXF para o Gephi
28/09/2026	083bdd8	Gerador de prints dos testes do menu
28/09/2026	bc94c05	Recorta a matriz de adjacência do print h) em janela legível de 20x20
28/09/2026	3273300	Corrige geração do PNG com kaleido em caminhos com espaço e ajusta rótulos clipados
28/09/2026	72d98ed	Guia de apresentação
28/09/2026	6cddf8f	README do projeto
28/09/2026	—	Relatório da Parte 2

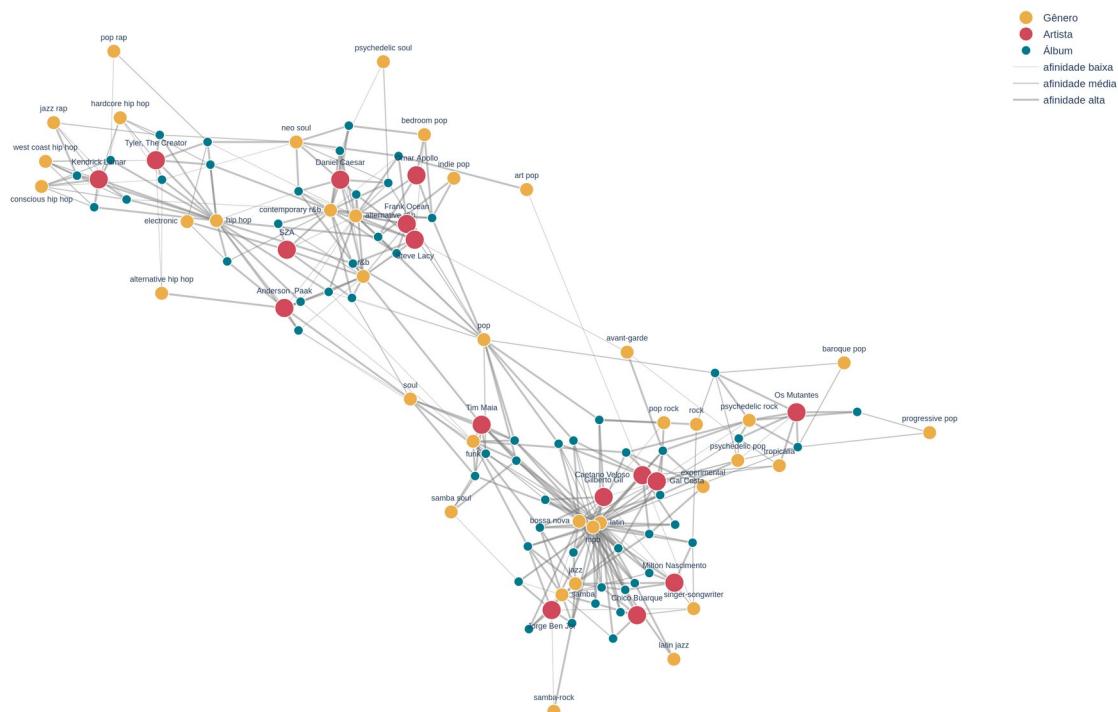
Fonte: autoria própria.

5.4 Visualização do grafo

O grafo foi desenhado com a biblioteca Plotly a partir do próprio grafo.txt, lido com a classe da aplicação; a disposição dos vértices usa o algoritmo de molas (spring layout) do NetworkX com semente fixa. A cor indica o tipo do vértice e a espessura da aresta, a afinidade (Figura 1). O mesmo grafo, com as mesmas posições e cores, foi exportado em formato GEXF e aberto no Gephi, uma das ferramentas indicadas pela disciplina (Figura 2). Uma versão interativa, com o nome e o grau de cada vértice ao passar o mouse, está em [visualizacao/grafico_interativo.html](#).

Figura 1 – Grafo tripartido álbum–artista–gênero (109 vértices, 349 arestas): artistas em vermelho, álbuns em azul e gêneros em amarelo

Grafo tripartido álbum–artista–gênero (109 vértices, 349 arestas) — dados: MusicBrainz



Fonte: autoria própria, dados do MusicBrainz.

Figura 2 – Grafo no Gephi (arquivo visualizacao/grafos.gexf)

[Inserir aqui o print do grafo no Gephi — visualizacao/grafos.gexf]

Fonte: autoria própria, dados do MusicBrainz.

A figura já evidencia a estrutura em duas regiões: na parte superior, o R&B e o hip-hop, organizados em torno de gêneros como hip hop, alternative r&b e contemporary r&b; na inferior, a MPB, concentrada em mpb, latin, bossa nova e samba. Entre as duas, poucos gêneros – soul, funk, pop e r&b – fazem a ligação.

5.5 Análise do grafo

A Tabela 10 reúne as medidas gerais do grafo, todas calculadas pelo script gerador deste relatório a partir de dados/grafos.txt.



Tabela 10 – Medidas gerais do grafo

Medida	Valor
Tipo do grafo	2 – não orientado com peso na aresta
Número de vértices (n)	109 (16 artistas, 56 álbuns, 37 gêneros)
Número de arestas (m)	349 (56 álbum-artista, 227 álbum-gênero, 66 artista-gênero)
Grau médio ($2m/n$)	6,40
Densidade $2m/(n(n-1))$	0,0593
Máximo de arestas possível no grafo tripartido	3560 (ocupação de 9,8%)
Componentes conexas	1 – grafo CONEXO
Diâmetro (em número de arestas)	8 (entre [GEN] conscious hip hop e [GEN] latin jazz)
Distância média entre pares (arestas)	3,40
Arestas de gênero com custo 0,00	153 de 293

Fonte: autoria própria.

Tabela 11 – Grau dos vértices por tipo

Tipo	Quantidade	Grau médio	Grau mínimo	Grau máximo	Vértice de grau máximo
Artista	16	7,62	4	9	[ART] Kendrick Lamar
Álbum	56	5,05	3	6	[ALB] IGOR — Tyler, The Creator
Gênero	37	7,92	2	30	[GEN] mpb

Fonte: autoria própria.

Tabela 12 – Os cinco gêneros de maior grau

Gênero	Grau	Ligações com R&B/hip-hop	Ligações com MPB
mpb	30	0	30
latin	29	0	29
hip hop	18	18	0
alternative r&b	17	17	0
contemporary r&b	17	17	0

Fonte: autoria própria.

O grafo é **conexo**: existe um caminho entre quaisquer dois vértices, ou seja, partindo de qualquer álbum é possível chegar a qualquer outro, o que é pré-requisito para a trilha de descoberta da próxima etapa. Com densidade 0,0593, é esparso: das 5886 ligações possíveis entre pares de vértices, só 349 existem – mesmo considerando apenas os pares permitidos pela tripartição, a ocupação é de 9,8%. O diâmetro de 8 arestas e a distância média de 3,40 arestas indicam que as duas cenas, embora separadas, ficam a poucos passos uma da outra.

Os gêneros de maior grau são "hubs" de uma única cena (Tabela 12): mpb e latin concentram a MPB, enquanto hip hop e os gêneros de r&b concentram a outra cena. Isso confirma que a classificação da comunidade separa bem as duas cenas. A pergunta mais interessante para a descoberta é, então, **quais vértices ligam as duas cenas**. Um gênero foi considerado ponte quando tem vizinhos (artistas ou álbuns) das duas cenas; dos 37 gêneros, 6 são pontes, 14 são exclusivos do R&B/hip-hop e 17 exclusivos da MPB (Tabela 13).

Tabela 13 – Gêneros-ponte entre as duas cenas (ordenados pelo menor lado)

Gênero	Grau	Vizinhos R&B/hip-hop	Vizinhos MPB
pop	15	8	7
funk	11	3	8
soul	7	3	4
r&b	16	15	1
avant-garde	3	1	2
art pop	2	1	1

Fonte: autoria própria.



Para identificar os artistas que sustentam essas pontes, somou-se a afinidade (1 – peso) das arestas que ligam o artista e seus álbuns a gêneros-ponte. A Tabela 14 mostra os três primeiros de cada cena.

Tabela 14 – Artistas-ponte: ligação do artista e de seus álbuns com gêneros-ponte

Artista	Cena	Arestas	Soma das afinidades	Gêneros-ponte alcançados
Anderson .Paak	R&B/hip-hop	10	5,14	funk, pop, r&b, soul
Steve Lacy	R&B/hip-hop	5	4,00	avant-garde, pop, r&b
SZA	R&B/hip-hop	5	2,67	pop, r&b, soul
Tim Maia	MPB	12	11,50	funk, pop, r&b, soul
Gal Costa	MPB	3	3,00	avant-garde, funk, pop
Gilberto Gil	MPB	3	2,50	funk, pop

Fonte: autoria própria.

Do lado brasileiro, Tim Maia é a principal ponte: sua obra do início dos anos 1970, marcada pelo soul e pelo funk norte-americanos, liga a MPB aos gêneros da outra cena. Do lado norte-americano, Anderson .Paak cumpre o papel simétrico, alcançando os gêneros-ponte funk, pop, r&b e soul. O resultado é coerente com a história da música e mostra que o modelo captura relações reais a partir apenas dos votos de gênero.

Como prévia da próxima etapa, a busca em largura encontrou um caminho com o menor número de arestas (3) entre [ALB] Blonde — Frank Ocean e [ALB] Fina estampa — Caetano Veloso: [ALB] Blonde — Frank Ocean → [GEN] art pop (custo 0,33) → [ART] Caetano Veloso (custo 0,90) → [ALB] Fina estampa — Caetano Veloso (custo 0,00), de custo total 1,23. A busca em largura ignora os pesos; o algoritmo de Dijkstra, na próxima etapa, poderá preferir um caminho mais longo, porém de menor custo total, isto é, formado por ligações mais fortes.

Análise crítica e limitações.

- **Viés de votos:** os gêneros refletem quem vota no MusicBrainz, comunidade de perfil majoritariamente anglófono. Os álbuns selecionados do R&B/hip-hop somam em média 14,9 votos de gênero, contra 6,3 nos álbuns de MPB.
- **Poucos votos em álbuns brasileiros antigos:** 45 dos 244 álbuns de estúdio da MPB coletados não têm nenhum gênero votado (contra 10 de 43 no R&B/hip-hop). Com poucos votos, vários gêneros empatam no máximo e recebem custo 0,00: isso ocorre em 90 de 129 arestas álbum–gênero da MPB e em 40 de 98 do R&B/hip-hop, o que reduz o poder de discriminação dos pesos na MPB.
- **Corte em cinco gêneros com desempate alfabético:** quando há empate, gêneros igualmente votados podem ficar de fora apenas pela ordem alfabética (por exemplo, soul no álbum Tim Maia de 1972).
- **Recorte curado:** os 16 artistas foram escolhidos pelo autor, o que determina as cenas e as pontes encontradas; o grafo não representa o universo musical, e sim duas cenas escolhidas para tornar a descoberta entre elas o problema central.
- **Nomes de gêneros em inglês:** a lista oficial do MusicBrainz usa nomes em inglês (latin, singer-songwriter), inclusive para gêneros brasileiros, e alguns gêneros se sobrepõem (mpb e latin, r&b e contemporary r&b).
- **Retrato de uma data:** os dados são um retrato da base na data da coleta; votos novos podem alterar os pesos, por isso os JSON brutos foram versionados para que o resultado seja reproduzível.



5.6 Testes

A suíte de testes automatizados (unittest) tem 22 testes e foi executada pelo script gerador deste relatório, com resultado **todos aprovados**. A Tabela 15 descreve o que cada arquivo verifica.

Tabela 15 – Testes automatizados

Arquivo	Testes	O que verifica
testes/test_app.py	6	Menu completo simulando a entrada do usuário: título, operação sem grafo, fluxo a–j, aresta de mesmo tipo recusada, índice fora do intervalo e fim da entrada.
testes/test_arquivo_grafo.py	5	Ida e volta ler → gravar → ler idêntica (também após remoções), formato do texto, recusa de tipo diferente de 2 e formatação.
testes/test_grafo_matriz.py	7	Inserção e simetria, peso 0,0 como aresta válida, recusa de laço/repetida/mesmo tipo/peso inválido, remoção de aresta, remoção de vértice com reindexação, componentes e tipo do vértice.
testes/test_montar_grafo.py	4	Cálculo das afinidades, construção do grafo, desambiguação de álbuns homônimos e mínimos do grafo real ($n \geq 80$, $m \geq 200$, tipo 2, tripartição).
Total	22	

Fonte: autoria própria.

Além dos testes automatizados, a aplicação foi exercitada com roteiros de uso do menu, com ao menos dois testes para cada opção (21 testes ao todo, cobrindo as opções a, b, c, d, e, f, g, h, i, j), incluindo casos válidos e entradas inválidas. As capturas foram produzidas pelo script ferramentas/gerar_prints.py, que executa a aplicação real com as entradas do roteiro, ecoa cada entrada como em um terminal e registra a saída completa em imagem; todas as operações são feitas sobre cópias temporárias do grafo.txt. Saídas muito longas aparecem truncadas. As Figuras 3 a 23 mostram os testes.

Figura 3 – Opção a) — teste 1: leitura do arquivo padrão dados/grafos.txt (109 vértices, 349 arestas)

```
python src/app.py

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Opção: a
Caminho do arquivo [Enter = /tmp/grafos_prints_d4wl2k2p/a1/grafos.txt]:
Grafo lido: 109 vértices, 349 arestas.

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.



Figura 4 – Opção a) — teste 2: leitura de um arquivo inexistente é tratada com mensagem de erro

```
python src/app.py

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Opção: a
Caminho do arquivo [Enter = /tmp/grafo_prints_d4wl2k2p/a2/grafo.txt]: inexistente.txt
Não foi possível ler o arquivo: [Errno 2] No such file or directory: 'inexistente.txt'

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.



Figura 5 – Opção b) — teste 1: após inserir um gênero novo, o grafo é gravado no caminho padrão (Enter)

```
python src/app.py

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Opção: a
Caminho do arquivo [Enter = /tmp/grafo_prints_d4wl2k2p/b1/grafo.txt]:
Grafo lido: 109 vértices, 349 arestas.

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Opção: c
Tipo do vértice (1 = álbum, 2 = artista, 3 = gênero): 3
Nome: neo-soul teste
Vértice 109 inserido: [GEN] neo-soul teste

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Opção: b
Caminho do arquivo [Enter = /tmp/grafo_prints_d4wl2k2p/b1/grafo.txt]:
Grafo gravado em /tmp/grafo_prints_d4wl2k2p/b1/grafo.txt (110 vértices, 349 arestas)

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.

Figura 6 – Opção b) — teste 2: gravação do grafo num caminho alternativo informado pelo usuário

```
python src/app.py

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Opção: a
Caminho do arquivo [Enter = /tmp/grafo_prints_d4wl2k2p/b2/grafo.txt]:
Grafo lido: 109 vértices, 349 arestas.

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Opção: b
Caminho do arquivo [Enter = /tmp/grafo_prints_d4wl2k2p/b2/grafo.txt]: /tmp/grafo_prints_d4wl2k2p/b2/copia_alvo.t
Grafo gravado em /tmp/grafo_prints_d4wl2k2p/b2/copia_alvo.txt (109 vértices, 349 arestas).

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.



Figura 7 – Opção c) — teste 1: inserção de um novo vértice do tipo álbum

```
python src/app.py

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====

a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação

Opção: a
Caminho do arquivo [Enter = /tmp/grafo_prints_d4wl2k2p/c1/grafo.txt]:
Grafo lido: 109 vértices, 349 arestas.

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====

a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação

Opção: c
Tipo do vértice (1 = álbum, 2 = artista, 3 = gênero): 1
Nome: Clube da Esquina 2 – Milton Nascimento
Já existe o vértice [ALB] Clube da Esquina 2 – Milton Nascimento.

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====

a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação

Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.

Figura 8 – Opção c) — teste 2: tipo de vértice inválido ("9") é recusado

```
python src/app.py

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====

a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação

Opção: a
Caminho do arquivo [Enter = /tmp/grafo_prints_d4wl2k2p/c2/grafo.txt]:
Grafo lido: 109 vértices, 349 arestas.

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====

a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação

Opção: c
Tipo do vértice (1 = álbum, 2 = artista, 3 = gênero): 9
Tipo inválido. Operação cancelada.

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====

a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação

Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.



Figura 9 – Opção d) — teste 1: aresta válida entre o novo gênero inserido e "[ART] Frank Ocean" (tipos diferentes)

```
python src/app.py

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====

a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação

Opção: a
Caminho do arquivo [Enter = /tmp/grafo_prints_d4wl2k2p/d1/grafo.txt]:
Grafo lido: 109 vértices, 349 arestas.

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====

a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação

Opção: c
Tipo do vértice (1 = álbum, 2 = artista, 3 = gênero): 3
Nome: gênero teste
Vértice 109 inserido: [GEN] gênero teste

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====

a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação

Opção: d
Vértice v: 109
Vértice w: 0
Peso (custo de descoberta, 0 a 1): 0.4
Aresta inserida: [GEN] gênero teste <-> [ART] Frank Ocean (peso 0.40)

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====

a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação

Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.

Figura 10 – Opção d) — teste 2: peso fora de [0,1] ("2") é rejeitado e, em seguida, a aresta entre "[ART] Frank Ocean" e "[ART] Tyler, The Creator" (mesmo tipo ART) é recusada pelo grafo tripartido

```
python src/app.py

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====

a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação

Opção: a
Caminho do arquivo [Enter = /tmp/grafo_prints_d4wl2k2p/d2/grafo.txt]:
Grafo lido: 109 vértices, 349 arestas.

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====

a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação

Opção: d
Vértice v: 0
Vértice w: 1
Peso (custo de descoberta, 0 a 1): 2
O peso (custo de descoberta) deve estar entre 0 e 1.
Peso (custo de descoberta, 0 a 1): 0.2
Aresta não inserida: já existe, é um laço ou liga vértices do mesmo tipo (o grafo é tripartido: álbum, artista e gênero)

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====

a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação

Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.



Figura 11 – Opção e) — teste 1: remoção do vértice de gênero "[GEN] alternative hip hop" e das 3 aresta(s) incidente(s)

```
python src/app.py

DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conexão do grafo
j) Encerrar a aplicação
Opção: a
Caminho do arquivo [Enter = /tmp/grafo_prints_d4wl2k2p/e1/grafo.txt]:
Grafo lido: 109 vértices, 349 arestas.

DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conexão do grafo
j) Encerrar a aplicação
Opção: e
Vértice a remover: 72
Removido [GEN] alternative hip hop e 3 aresta(s) incidente(s). Os vértices após 72 tiveram o índice reduzido em 1.

DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conexão do grafo
j) Encerrar a aplicação
Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.

Figura 12 – Opção e) — teste 2: índice fora do intervalo válido ("999") é rejeitado; Enter vazio cancela a remoção

```
python src/app.py

DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conexão do grafo
j) Encerrar a aplicação
Opção: a
Caminho do arquivo [Enter = /tmp/grafo_prints_d4wl2k2p/e2/grafo.txt]:
Grafo lido: 109 vértices, 349 arestas.

DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conexão do grafo
j) Encerrar a aplicação
Opção: e
Vértice a remover: 999
Valor fora do intervalo [0, 108].
Vértice a remover:
Operação cancelada.

DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conexão do grafo
j) Encerrar a aplicação
Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.



Figura 13 – Opção f) — teste 1: remoção da aresta existente entre "[ALB] Blonde — Frank Ocean" e "[GEN] alternative r&b"

```
python src/app.py

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Opção: a
Caminho do arquivo [Enter = /tmp/grafo_prints_d4wl2k2p/f1/grafo.txt]:
Grafo lido: 109 vértices, 349 arestas.

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Opção: f
Vértice v: 16
Vértice w: 73
Aresta removida: [ALB] Blonde — Frank Ocean <-> [GEN] alternative r&b

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.

Figura 14 – Opção f) — teste 2: tentativa de remover uma aresta inexistente entre "[ART] Frank Ocean" e "[ART] Tyler, The Creator" (dois artistas nunca ficam ligados) é tratada

```
python src/app.py

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Opção: a
Caminho do arquivo [Enter = /tmp/grafo_prints_d4wl2k2p/f2/grafo.txt]:
Grafo lido: 109 vértices, 349 arestas.

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Opção: f
Vértice v: 0
Vértice w: 1
Não existe aresta entre esses vértices.

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.



Figura 15 – Opção g) — teste 1: exibição formatada do conteúdo do arquivo padrão (saída truncada para caber na imagem)

```
python src/app.py

DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conexidade do grafo
j) Encerrar a aplicação

Opção: g
Caminho do arquivo [Enter = /tmp/grafo_prints_d4wl2k2p/g1/grafo.txt]:
Arquivo: /tmp/grafo_prints_d4wl2k2p/g1/grafo.txt
Tipo do grafo: 2 - não orientado com peso na aresta
Vértices: 109  Arestas: 349

Nr  Tipo  Rótulo
-----
0  ART   Frank Ocean
1  ART   Tyler, The Creator
2  ART   Daniel Caesar
3  ART   Omar Apollo
4  ART   SZA
5  ART   Steve Lacy
6  ART   Kendrick Lamar
7  ART   Anderson .Paak
8  ART   Caetano Veloso
9  ART   Gilberto Gil
10 ART   Chico Buarque
11 ART   Gal Costa
12 ART   Milton Nascimento
13 ART   Jorge Ben Jor
14 ART   Tim Maia
15 ART   Os Mutantes
16 ALB   Blonde - Frank Ocean
17 ALB   channel ORANGE - Frank Ocean
18 ALB   DON'T TAP THE GLASS - Tyler, The Creator
19 ALB   IGOR - Tyler, The Creator
20 ALB   CIRCUMSPIZA - Tyler, The Creator
21 ALB   Wolf - Tyler, The Creator
22 ALB   Freudian - Daniel Caesar
23 ALB   NEVER ENOUGH - Daniel Caesar
24 ALB   Case Study 01 - Daniel Caesar
25 ALB   Son of Sperm - Daniel Caesar
26 ALB   God Said No - Omar Apollo
... (439 linhas omitidas) ...
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conexidade do grafo
j) Encerrar a aplicação

Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.

Figura 16 – Opção g) — teste 2: leitura do conteúdo de um arquivo inexistente é tratada com erro

```
python src/app.py

DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conexidade do grafo
j) Encerrar a aplicação

Opção: g
Caminho do arquivo [Enter = /tmp/grafo_prints_d4wl2k2p/g2/grafo.txt]: inexistente.txt
Não foi possível mostrar o arquivo: [Errno 2] No such file or directory: 'inexistente.txt'

DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conexidade do grafo
j) Encerrar a aplicação

Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.



Figura 19 – Opção h) — teste 3: exibir o grafo sem antes carregá-lo (opção a) é bloqueado

```
python src/app.py

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Opção: h
Nenhum grafo carregado. Use a opção a) primeiro.

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.

Figura 20 – Opção i) — teste 1: o grafo real é CONEXO (1 componente com os 109 vértices)

```
python src/app.py

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Opção: i
Caminho do arquivo [Enter = /tmp/grafo_prints_d4wl2k2p/11/grafo.txt]:
Grafo lido: 109 vértices, 349 arestas.

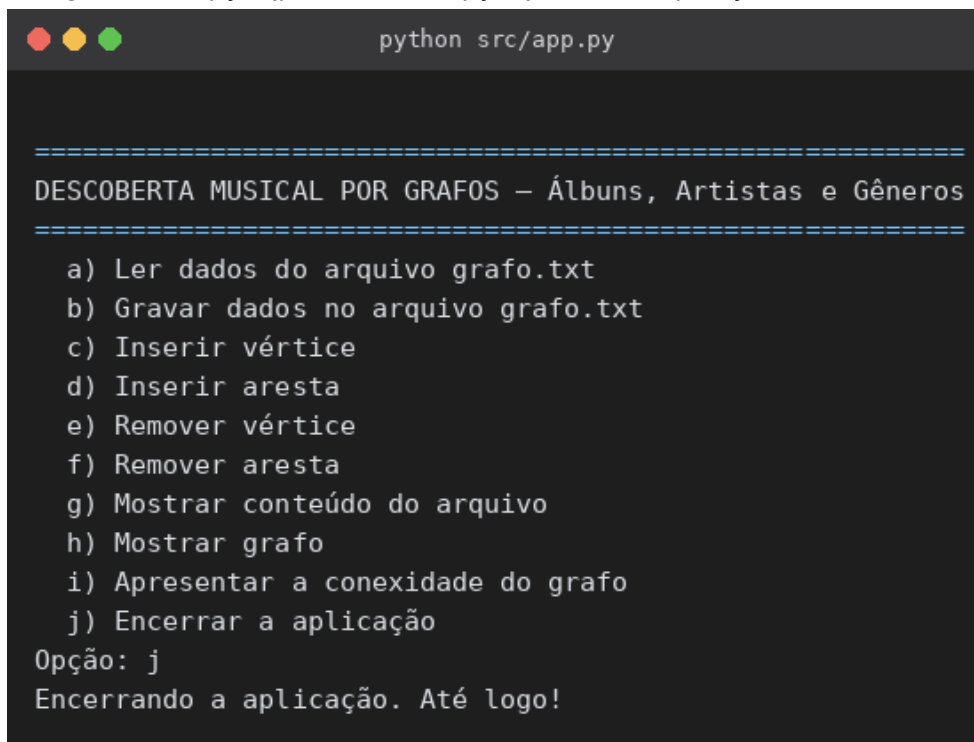
=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Opção: i
O grafo é CONEXO (1 componente(s) conecta(s)).
Componente 1: 109 vértice(s) - [ART] Frank Ocean; [ART] Tyler, The Creator; [ART] Daniel Caesar; [ART] Omar Apollo; [ART] SZA; [ART] Steve Lacy; [ART] Kendrick Lamar; [ART] Anderson .Paak ...
Obs.: grafo NÃO orientado - as categorias CB-C3, o algoritmo FCONE e o grafo reduzido se aplicam a grafos orientados.

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.

[illegible]

Figura 22 – Opção j) — teste 1: a opção j encerra a aplicação imediatamente



Fonte: autoria própria.



Figura 23 – Opção j) — teste 2: uma opção inválida ("x") é rejeitada antes de encerrar com j

```
python src/app.py

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Opção: x
Opção inválida.

=====
DESCOBERTA MUSICAL POR GRAFOS – Álbuns, Artistas e Gêneros
=====
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo
i) Apresentar a conectividade do grafo
j) Encerrar a aplicação
Opção: j
Encerrando a aplicação. Até logo!
```

Fonte: autoria própria.

6 OBJETIVOS DE DESENVOLVIMENTO SUSTENTÁVEL

O projeto se relaciona com três Objetivos de Desenvolvimento Sustentável da Agenda 2030 (IBGE, [2026]):

- **ODS 9 – Indústria, inovação e infraestrutura:** desenvolve uma ferramenta computacional aberta, baseada em dados públicos e software livre, que modela e analisa uma rede de conhecimento musical; o código e os dados ficam disponíveis para reuso, contribuindo para a capacidade tecnológica e a inovação.
- **ODS 4 – Educação de qualidade:** a trilha de descoberta entre álbuns funciona como instrumento de acesso e formação cultural, apresentando ao ouvinte obras e gêneros que ele não conhece a partir do que já conhece; o próprio projeto é também material didático de Teoria dos Grafos, com dados reais.
- **ODS 11 (meta 11.4) – Proteger e salvaguardar o patrimônio cultural:** a MPB é parte do patrimônio cultural brasileiro. O modelo liga discos históricos, como os de Tim Maia, Gal Costa e Os Mutantes, a artistas contemporâneos, o que favorece que esse repertório seja redescoberto por novos públicos; a análise também evidencia a sub-representação dos álbuns brasileiros antigos nas bases abertas, um problema de preservação digital.

7 PRÓXIMAS ETAPAS

No segundo bimestre, a função central do protótipo será a **trilha de descoberta**: dados um álbum de partida e um álbum de chegada, o algoritmo de Dijkstra, implementado sobre a mesma matriz de adjacência, encontrará o caminho de menor custo total, mostrando os artistas e gêneros intermediários e o custo de cada passo. Como os pesos estão em $[0, 1]$, o requisito de pesos não



negativos é satisfeito. Estão previstos ainda: nova opção no menu para a trilha, comparação entre o caminho de menor custo e o de menor número de arestas, e a ampliação do conjunto de artistas semente para testar a robustez dos gêneros-ponte encontrados. Usuários, detecção de comunidades e recomendação personalizada permanecem fora do escopo da disciplina.

REFERÊNCIAS

BLONDEL, V. D.; GUILLAUME, J.-L.; LAMBIOTTE, R.; LEFEBVRE, E. Fast unfolding of communities in large networks. **Journal of Statistical Mechanics: Theory and Experiment**, v. 2008, n. 10, p. P10008, 2008. DOI: 10.1088/1742-5468/2008/10/P10008.

BONDY, J. A.; MURTY, U. S. R. **Graph theory**. New York: Springer, 2010.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). **Diário Oficial da União**, Brasília, DF, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 28 set. 2026.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Introduction to algorithms**. 2. ed. Cambridge, MA: MIT Press, 2001.

DATTA, H.; KNOX, G.; BRONNENBERG, B. J. Changing their tune: how consumers' adoption of online streaming affects music consumption and discovery. **Marketing Science**, v. 37, n. 1, p. 5-21, 2018. DOI: 10.1287/mksc.2017.1051.

DIESTEL, R. **Graph theory**. 4. ed. New York: Springer, 2010.

EKSOMBATCHAI, C. et al. Pixie: a system for recommending 3+ billion items to 200+ million users in real-time. In: WORLD WIDE WEB CONFERENCE, 2018, Lyon. **Proceedings [...]**. New York: ACM, 2018. p. 1775-1784. DOI: 10.1145/3178876.3186183.

IBGE – INSTITUTO BRASILEIRO DE GEOGRAFIA E ESTATÍSTICA. **Indicadores Brasileiros para os Objetivos de Desenvolvimento Sustentável**. Rio de Janeiro: IBGE, [2026]. Disponível em: <https://odsbrasil.gov.br/>. Acesso em: 28 set. 2026.

LIKA, B.; KOLOMVATSOS, K.; HADJIEFTHYMIADES, S. Facing the cold start problem in recommender systems. **Expert Systems with Applications**, v. 41, n. 4, pt. 2, p. 2065-2073, 2014. DOI: 10.1016/j.eswa.2013.09.005.

MUSICBRAINZ. **MusicBrainz API**. [S. l.]: MetaBrainz Foundation, [2026]. Disponível em: https://musicbrainz.org/doc/MusicBrainz_API. Acesso em: 28 set. 2026.

SHARMA, A.; JIANG, J.; BOMMANAVAR, P.; LARSON, B.; LIN, J. GraphJet: real-time content recommendations at Twitter. **Proceedings of the VLDB Endowment**, v. 9, n. 13, p. 1281-1292, 2016. DOI: 10.14778/3007263.3007267.

SHI, C.; LI, Y.; ZHANG, J.; SUN, Y.; YU, P. S. A survey of heterogeneous information network analysis. **IEEE Transactions on Knowledge and Data Engineering**, v. 29, n. 1, p. 17-37, 2017. DOI: 10.1109/TKDE.2016.2598561.

SZWARCFITER, J. L. **Grafos e algoritmos computacionais**. 2. ed. Rio de Janeiro: Campus, 1988.

UGANDER, J.; KARRER, B.; BACKSTROM, L.; MARLOW, C. **The anatomy of the Facebook social graph**. arXiv:1111.4503, 2011. Disponível em: <https://arxiv.org/abs/1111.4503>. Acesso em: 28 set. 2026.



VAN DEN OORD, A.; DIELEMAN, S.; SCHRAUWEN, B. Deep content-based music recommendation. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 26., 2013. **Proceedings** [...]. 2013. p. 2643-2651.

YING, R.; HE, R.; CHEN, K.; EKSOMBATCHAI, P.; HAMILTON, W. L.; LESKOVEC, J. Graph convolutional neural networks for web-scale recommender systems. In: ACM SIGKDD INTERNATIONAL CONFERENCE ON KNOWLEDGE DISCOVERY AND DATA MINING, 24., 2018, London. **Proceedings** [...]. New York: ACM, 2018. DOI: 10.1145/3219819.3219890.

APÊNDICE A – REPOSITÓRIO NO GITHUB

Código-fonte, dados, testes, figuras e este relatório estão disponíveis, com acesso público, em: <https://github.com/luisfelipesdn12/grafos-musical-tg>

Para reproduzir os resultados: instalar as dependências (pip install -r requirements.txt); executar a aplicação (python3 src/app.py); rodar os testes (python3 -m unittest discover -s testes); refazer a coleta e o grafo, se desejado (python3 coleta/coletar_musicbrainz.py e python3 coleta/montar_grafos.py); regenerar a visualização (python3 visualizacao/visualizar.py) e este relatório (python3 ferramentas/gerar_relatorio.py).